

## Session 07

---

### Topic Overview

This session covered **Snowflake Time Travel & Fail-safe, retention period behavior**, and a **real-world data migration flow** from source systems (SQL Server) to Snowflake, including **DDL conversion, CSV extraction**, and **manual data loading**.

---

### 1. Snowflake Time Travel – Retention Periods

Time Travel allows accessing historical data (deleted/updated data) within a defined retention period.

#### Retention by Edition

##### Snowflake Edition    Max Time Travel Retention

Standard Edition    1 Day

Enterprise & Above Up to 90 Days

#### Example

```
ALTER TABLE EMP SET DATA_RETENTION_TIME_IN_DAYS = 90;
```

---

### 2. Fail-safe Period (Fixed – 7 Days)

- Fail-safe starts **immediately after Time Travel ends**
- Duration is **fixed: 7 days**
- Data recovery during fail-safe is handled **only by Snowflake Support**

#### Maximum Recovery Window

- 90 days (Time Travel)
  - 7 days (Fail-safe)
  - **Total = 97 days**
- 

### 3. Decreasing Retention Period – Behavior

#### Example

Retention reduced from **10 days → 1 day**

#### Result

- Only **1 day** remains in Time Travel
- Data from **day 2 to day 8** moves to Fail-safe

- Older fail-safe data is overwritten

⚠ Movement happens via **background process**, not immediately

---

#### 4. Increasing Retention Period – Important Rules

- Data in Fail-safe **never returns** to Time Travel
- Increasing retention does **not** restore older data

##### Example

- Retention increased from 1 day → 3 days on Oct 10
- Oct 10: still 1 day available
- Oct 11: 2 days available
- Oct 12: 3 days available

Manual backup is required for guaranteed recovery

---

#### 5. Retention Period Limits

- Maximum retention for permanent tables = **90 days**
  - Cannot increase beyond 90 days
- 

#### 6. Time Travel Example Timeline

- Table created: **Oct 1, 8:00 AM**
- Retention: **1 day**
- After Oct 2: data moves to Fail-safe
- After Oct 8: data permanently lost

Data beyond Time Travel + Fail-safe **cannot be recovered**

---

#### 7. Typical Snowflake Project Flow

1. Source Databases (Oracle, SQL Server, MySQL)
  2. Extract data as files (CSV, JSON, Parquet)
  3. Store files in **Data Lake** (Raw / Enriched buckets)
  4. Load data into **Snowflake**
  5. Build reports using **Power BI / Tableau**
-

## 8. Schema Migration Process

### Step 1: Table Creation

- Recreate source tables in Snowflake
- Use source DDL scripts

### If No Source Access

- DDL scripts provided via **Jira ticket**
- 

## 9. DDL Conversion – SnowConvert Tool

### Why Conversion is Required

- Some source data types are not supported
- Example: SQL Server MONEY datatype

### Tool

- **SnowConvert** (Free)
- Converts code from:
  - SQL Server
  - Oracle
  - Teradata

### Example Conversion

#### SQL Server Snowflake

MONEY     NUMBER / DECIMAL

---

## 10. VARCHAR vs STRING in Snowflake

- STRING = VARCHAR(MAX)
- VARCHAR(50) enforces length

### Example

FIRST\_NAME STRING,

LAST\_NAME VARCHAR(50)

---

## 11. Creating Table in Snowflake (Demo)

```
CREATE TABLE PROD_DB.SF_SCHEMA.CUSTOMERS (  
  ID NUMBER,
```

```
FIRST_NAME STRING,  
LAST_NAME STRING,  
ADDRESS1 STRING,  
ADDRESS2 STRING  
);
```

---

## 12. Data Extraction from SQL Server (Manual)

### Tool Used

- SQL Server Export Wizard

### Steps

1. Select SQL Server source
  2. Choose Windows / SQL authentication
  3. Select database & table
  4. Choose Flat File Destination
  5. File type: CSV
  6. Column delimiter: Comma
  7. Row delimiter: CR/LF (New Line)
- 

## 13. Understanding CSV Files

- Comma Separated Values
  - Each row on a new line
  - Best viewed in **Notepad / Notepad++**
- 

## 14. Loading CSV into Snowflake (UI Method)

### Steps

1. Select table in Snowflake UI
2. Click **Load Data**
3. Choose CSV file
4. Specify column delimiter
5. Enable **Skip Header**
6. Load data

---

## 15. Why Skip Header is Mandatory

- Header contains column names
  - Column names are strings
  - Loading into numeric columns causes failure
- 

## 16. Manual vs Real-Time Project Approach

### Manual (Training) Real Project

|                |                     |
|----------------|---------------------|
| UI based load  | Automated pipelines |
| Local CSV      | Data lake storage   |
| Manual scripts | Tasks & scheduling  |

---

### Key Takeaways

- Time Travel max = 90 days
  - Fail-safe is fixed = 7 days
  - Fail-safe data never returns to Time Travel
  - Increasing retention doesn't restore old data
  - SnowConvert simplifies schema migration
  - STRING = VARCHAR MAX
  - Skip header avoids datatype errors
- 

### Next Topics

- COPY command automation
- Snowflake Tasks
- Error handling in production pipelines

## How do you connect SQL Server to Snowflake manually?

### Answer:

In real projects, SQL Server is **not directly connected** to Snowflake.

Instead, data is **manually extracted from SQL Server as files (CSV)** and then **loaded into Snowflake**.

I'll explain the **end-to-end manual process step by step**, including navigation.

---

### STEP 1: Understand the Manual Approach

Snowflake **does not connect directly** to SQL Server like a database link.

Manual flow:

SQL Server → CSV File → Snowflake Table

This approach is mainly used for:

- POC
  - Training
  - One-time loads
  - Small datasets
- 

### STEP 2: Extract Data from SQL Server (CSV Creation)

#### Navigation in SQL Server

1. Open **SQL Server Management Studio (SSMS)**
  2. Connect to SQL Server using:
    - Windows Authentication or
    - SQL Server Authentication
  3. Expand:
  4. Databases → <Your Database> → Tables
  5. Right-click the required table
    - 👉 **Tasks → Export Data**
- 

#### SQL Server Export Wizard

1. **Data Source**
  - Source: SQL Server
  - Authentication: Windows / SQL
  - Database: Select DB

## 2. Destination

- Destination: **Flat File Destination**
- File Name:  
customers\_09\_12\_2025\_1000.csv
- Format: Delimited

## 3. Delimiter Settings

- Column delimiter: , (Comma)
  - Row delimiter: CR/LF (New Line)
4. Select table (example: dim\_customer)
  5. Finish → CSV file is generated

✚ Result: Data is now available as a **CSV file**

---

## STEP 3: Prepare Snowflake Table (Schema Migration)

### Convert SQL Server DDL

- SQL Server datatypes like MONEY are not supported
- Use:
  - Manual conversion **OR**
  - **SnowConvert** tool

### Example Table Creation in Snowflake

```
CREATE TABLE PROD_DB.SF_SCHEMA.CUSTOMERS (  
  CUSTOMER_ID NUMBER,  
  FIRST_NAME STRING,  
  LAST_NAME STRING,  
  ADDRESS STRING,  
  CITY STRING  
);
```

✚ STRING works as VARCHAR(MAX) in Snowflake

---

#### STEP 4: Login to Snowflake UI

1. Open Snowflake Web Console
  2. Login with credentials
  3. Select:
  4. Database → Schema → Table
- 

#### STEP 5: Load CSV File into Snowflake (Manual UI Method)

##### Navigation

1. Go to **Databases**
  2. Select:
  3. PROD\_DB → SF\_SCHEMA → CUSTOMERS
  4. Click **Load Data**
  5. Upload CSV file
- 

##### File Format Settings

- File type: CSV
- Field delimiter: ,
- Skip Header: ☒ **Yes**
- Encoding: UTF-8

##### ✦ Why Skip Header?

Because header contains column names (strings), which will fail when loading into numeric columns.

---

#### STEP 6: Load & Validate Data

1. Click **Load**
2. Snowflake shows:
  - Rows loaded
  - Load status
3. Preview table:

```
SELECT * FROM PROD_DB.SF_SCHEMA.CUSTOMERS;
```

☒ Data from SQL Server is now available in Snowflake

---



## STEP 7: Real Project vs Manual Method (Interview Point)

| Manual (Training) | Real Project          |
|-------------------|-----------------------|
| UI based load     | Automated pipelines   |
| Local CSV files   | Data Lake (S3 / ADLS) |
| One-time load     | Scheduled loads       |
| No tasks          | Snowflake Tasks       |

---

### Interview Summary (One-Line)

Manually, SQL Server is connected to Snowflake by exporting data as CSV files from SQL Server and loading them into Snowflake using the UI or COPY command after recreating the schema.

---

### Strong Interview Closing Line

Snowflake does not support direct SQL Server connections; manual connection always happens through file-based data movement.

These are **real interview-oriented** and frequently asked in Snowflake roles.

---

### 1. What is Time Travel in Snowflake?

Time Travel allows you to **access historical data** (deleted/updated rows or dropped tables) within a defined **retention period**.

---

### 2. What is the maximum Time Travel retention period in Snowflake?

- **Standard Edition:** 1 day
  - **Enterprise & higher:** Up to **90 days**
- 

### 3. What is Fail-safe in Snowflake and how long does it last?

Fail-safe is a **fixed 7-day recovery period** after Time Travel ends. Data can be recovered **only by Snowflake Support**, not by users.

---

### 4. What is the maximum total data recovery period in Snowflake?

- 90 days (Time Travel)
- 7 days (Fail-safe)

✓ **Total = 97 days**

---

### 5. What happens when you decrease the Time Travel retention period?

- Time Travel data reduces immediately
  - Extra historical data moves to **Fail-safe**
  - Movement happens via a **background process**
- 

### 6. What happens when you increase the retention period?

- Data in Fail-safe **does not move back** to Time Travel
  - New retention period is **not immediately available**
  - Data accumulates going forward only
-

### 7. Can data in Fail-safe be accessed using SQL?

✗ No

Fail-safe recovery is handled **only by Snowflake Support**

---

### 8. What is SnowConvert and why is it used?

SnowConvert is a **free Snowflake migration tool** used to:

- Convert DDL/code from SQL Server, Oracle, Teradata
  - Replace unsupported data types (e.g., MONEY → NUMBER/DECIMAL)
- 

### 9. What is the difference between STRING and VARCHAR in Snowflake?

- STRING = VARCHAR(MAX)
  - VARCHAR(n) enforces a **length limit**
- 

### 10. Why is “Skip Header” required while loading CSV files?

- Header contains column names (strings)
  - Loading into numeric columns causes **datatype errors**
  - Skipping header prevents load failure
- 

### 🔥 Bonus Interview Tip

**Fail-safe data never returns to Time Travel** — this is a very common **trap question**.